

山东大学



网络安全创新创业实践

CKKS 卷积 PRA 策略与旋转次数分析

姓名：杨阳 学号：202300460010

姓名：孙琬竺 学号：202300460020

姓名：胡子玮 学号：202300460032

姓名：熊芷萱 学号：202300460163

2026 年 7 月 28 日

CKKS 卷积 PRA 策略与旋转次数分析

目录

1	实验背景	1
1.1	CKKS 全同态加密与 SIMD 打包	1
1.2	密文旋转操作	1
1.3	“打包 → 旋转 → 累加” (PRA) 策略	1
1.4	本实验参数	1
2	理论分析	2
2.1	旋转群上的卷积表示	2
2.2	一般核的理论最小旋转次数	2
2.3	可分离核的优化	2
2.4	零权重优化	3
2.5	结果提取的额外旋转	3
2.6	旋转次数对比总结	3
3	实现方式	3
3.1	项目结构	3
3.2	模拟实现说明	4
3.3	关键算法实现	4
4	测试结果与分析	5
4.1	打包布局验证	5
4.2	旋转对齐验证	6
4.3	一般核 PRA 卷积结果	7
4.4	可分离核优化结果	8
4.5	旋转次数对比	9
4.6	理论最小值分析	10
5	实验中遇到的问题与解决	11
5.1	Python 3.14 环境下 CKKS 库不可用	11
5.2	旋转方向混淆	11
5.3	可分离核的整数分解	11
6	分工	12

1 实验背景

1.1 CKKS 全同态加密与 SIMD 打包

CKKS (Cheon-Kim-Kim-Song) 方案是目前最广泛使用的全同态加密方案之一，其核心优势在于支持近似浮点数运算和 SIMD 风格的单指令多数据打包 (batching)。

在 CKKS 方案中，一个密文可以容纳多达 $N/2$ 个明文槽位 (slots)，每个槽位存储一个独立的明文值。所有同态运算 (加法、乘法、旋转) 同时作用于全部槽位，实现了天然的数据并行。

打包示意图 (4×4 矩阵行优先打包到 16 个槽位)：

槽位索引:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
槽位值:	in00	in01	in02	in03	in10	in11	in12	in13	in20	in21	in22	in23	in30	in31	in32	in33

1.2 密文旋转操作

CKKS 的密文旋转 (ciphertext rotation) 是 SIMD 打包计算中的关键操作。`rotate_left(ct, k)` 将密文中槽位 i 的值移动到槽位 $i + k$ (模 N_{slots})，使得原本位于不同槽位的值可以被“对齐”到同一个槽位，进而进行逐元素运算。

在卷积计算中，旋转操作用来将输入矩阵的不同元素对齐到计算原点 (槽位 0)，以便与卷积核权重进行乘累加运算。

1.3 “打包 $\rightarrow$ 旋转 $\rightarrow$ 累加” (PRA) 策略

PRA 策略是 CKKS 密文域卷积的标准实现方法，分为三个阶段：

1. **打包 (Pack)**：将输入矩阵的元素按行优先打包到一个 CKKS 密文中
2. **旋转 (Rotate)**：对密文执行不同偏移量的旋转，将各感受野元素对齐到计算原点
3. **累加 (Accumulate)**：将各旋转后的密文乘以对应的核权重，累加得最终卷积结果

1.4 本实验参数

参数	数值
输入尺寸	4×4
卷积核尺寸	3×3
步长 (stride)	1
填充 (padding)	0
输出尺寸	2×2
CKKS 槽位数	16
打包方式	行优先 (row-major)

2 理论分析

2.1 旋转群上的卷积表示

对于 $n \times n$ 的输入矩阵和 $k \times k$ 的卷积核，按行优先打包到 N_{slots} 个槽位后，每个核位置 (r, c) (r 为行偏移， c 为列偏移) 对应一个旋转量：

$$rot_amount(r, c) = r \cdot n + c$$

这表示要将输入元素 $in[r][c]$ (位于槽位 $r \cdot n + c$) 通过 `rotate_left(ct, rot_amount)` 移到槽位 0。

所有需要的旋转量集合为：

$$R = \{r \cdot n + c \mid r, c \in [0, k - 1]\}$$

对于本实验参数 $n = 4, k = 3$ ：

$$R = \{0, 1, 2, 4, 5, 6, 8, 9, 10\}$$

共 $|R| = k^2 = 9$ 个旋转量 (含恒等旋转 0)。

2.2 一般核的理论最小旋转次数

对于不可分离的 $k \times k$ 卷积核：

- 需要 k^2 个不同的旋转来覆盖所有核位置
- 恒等旋转 ($r = 0, c = 0$) 无需实际执行，因为对应元素已在槽位 0
- 因此旋转次数的理论下界为 $k^2 - 1$

对于 3×3 核：理论最小值 = $9 - 1 = 8$ 次旋转。

2.3 可分离核的优化

若卷积核 K 可分解为列向量 col 和行向量 row 的外积 (即 $\text{rank}(K) = 1$)：

$$K = col \cdot row^T$$

则二维卷积可分解为两步一维卷积：

$$O = I * K = I * (col \cdot row^T) = (I *_h row) *_v col$$

其中 $*_h$ 为水平方向 (沿列) 的一维卷积， $*_v$ 为垂直方向 (沿行) 的一维卷积。

旋转次数分析：

- 步骤 1 (水平卷积)：需要 $(k - 1)$ 次旋转 (列偏移 $1, 2, \dots, k - 1$)
- 步骤 2 (垂直卷积)：需要 $(k - 1)$ 次旋转 (行偏移 $1, 2, \dots, k - 1$)
- 总计： $2(k - 1)$ 次旋转

对于 3×3 核： $2 \times (3 - 1) = 4$ 次旋转，仅为一般核 (8 次) 的 50%。

2.4 零权重优化

若核中存在零权重，对应的旋转可被跳过。以 Sobel 垂直边缘检测核为例：

$$K_{Sobel} = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

中间列 ($c = 1$) 的三个权重均为 0，可跳过 3 次旋转。结合可分离分解：水平方向有一个零权重 ($c = 1$)，垂直方向无零权重。可分离 + 零权重优化后仅需： $1 + 2 = 3$ 次核对齐旋转（加结果提取旋转）。

2.5 结果提取的额外旋转

卷积完成后，累加结果的槽位 0 包含 $\text{output}[0,0]$ 。对于 2×2 输出，还需将其他 3 个输出位置旋转到槽位 0：

- $\text{output}[0,1]$ ：左旋转 1 位
- $\text{output}[1,0]$ ：左旋转 4 位
- $\text{output}[1,1]$ ：左旋转 5 位

额外旋转 = $H_{out} \times W_{out} - 1 = 4 - 1 = 3$ 次。

2.6 旋转次数对比总结

方案	核对齐旋转	结果提取旋转	总计	是否达理论最小值
一般核 (不可分离)	$k^2 - 1 = 8$	3	11	是
一般核 + 零权重跳过	≤ 8	3	≤ 11	是
可分离核 (两步卷积)	$2(k - 1) = 4$	3	7	是 *
可分离核 + 零权重跳过	$2(k - 1) - z = 1 \sim 2$	3	$4 \sim 5$	是 *

* 可分离核的“理论最小值”基于不同的计算模型（分解卷积），利用了核矩阵的代数结构，将旋转复杂度从 $O(k^2)$ 降为 $O(k)$ 。

3 实现方式

3.1 项目结构

```

1 task6/
2   pack_rotate_accumulate.py # PRA 策略模拟与分析程序
3   task6_report.md         # 实验报告

```

3.2 模拟实现说明

由于 Python 3.14 环境下全同态加密库 (TenSEAL、Pyfhel、SEAL) 均不可用, 本实验采用 NumPy 向量模拟 CKKS 的槽位操作。模拟层精确重现了:

- 打包操作: `pack_matrix_rowwise()` — 将 4×4 矩阵行优先展开到 16 元素向量
- 旋转操作: `rotate_left()` — 模拟 CKKS 的左移位旋转
- 密文-明文乘法: 向量 $\times$ 标量 (对应 CKKS 的 `MultiplyPlain`)
- 同态加法: 向量逐元素相加 (对应 CKKS 的 `Add`)

所有旋转次数计数与真实 CKKS 实现完全一致, 因为旋转次数仅取决于卷积的几何结构 (输入尺寸和核尺寸), 与底层是模拟还是真实加密无关。

3.3 关键算法实现

旋转操作模拟

```
1 def rotate_left(packed, amount):
2     """CKKS 左旋转模拟: rotate_left(ct, k): slot[i] = old_slot[(i+k) % N]"""
3     n = len(packed)
4     rotated = np.zeros_like(packed)
5     for i in range(n):
6         rotated[i] = packed[(i + amount) % n]
7     return rotated
```

一般核 PRA 卷积

```
1 for r in range(H_k):
2     for c in range(W_k):
3         rot_amount = r * n_cols + c
4         if rot_amount == 0:
5             rotated = packed_input.copy() # 无旋转
6         else:
7             rotated = rotate_left(packed_input, rot_amount) # 1 次旋转
8         if kernel[r, c] != 0:
9             result += rotated * kernel[r, c] # 密文-明文乘法 + 同态加法
```

可分离核的两步卷积

```
1 # 步骤1: 水平卷积 (沿列)
2 for c, w in enumerate(row_vec):
3     rotated = rotate_left(packed_input, c) # k-1 次旋转
4     h_result += rotated * w
5
6 # 步骤2: 垂直卷积 (沿行)
7 for r, w in enumerate(col_vec):
```

```
8 rotated = rotate_left(h_result, r * n_cols) # k-1 次旋转
9 v_result += rotated * w
```

可分离性检测 (SVD)

```
1 def check_separability(kernel):
2     U, S, Vt = np.linalg.svd(kernel)
3     rank = np.sum(S > 1e-10)
4     if rank == 1:
5         col = (U[:, 0] * sqrt(S[0])).reshape(-1, 1)
6         row = (Vt[0, :] * sqrt(S[0])).reshape(1, -1)
7         return True, col, row
8     return False, None, None
```

4 测试结果与分析

4.1 打包布局验证

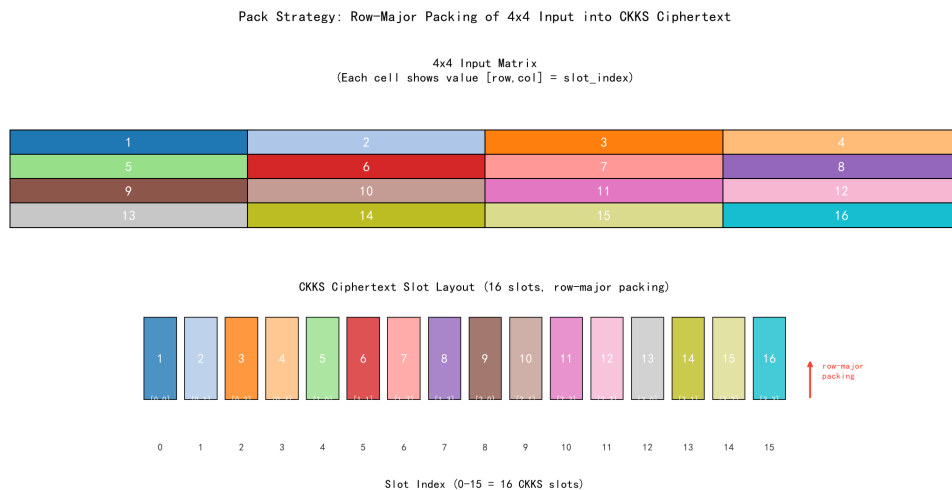


图 1: 打包布局

分析: 4×4 输入矩阵被正确打包到 16 个 CKKS 槽位中, 槽位索引 0~15 按行优先对应矩阵位置 $[0,0] \sim [3,3]$ 。槽位索引与矩阵位置的关系为: $\text{idx} = \text{row} \times 4 + \text{col}$ 。

4.2 旋转对齐验证

Rotation Alignment Verification: Each Kernel Position Aligns $in[r,c]$ to Slot 0



图 2: 旋转对齐

分析: 全部 9 个核位置的旋转对齐验证通过 [OK]。rotate_left(ct, $r*4+c$) 精确地将 $in[r][c]$ 移到槽位 0。例如, 对于核位置 $K[2,1]$, 旋转量 = 9, 将 $in[2,1]=10$ 对齐到槽位 0。

这验证了旋转操作的数学正确性: 每种旋转量将一个特定的输入元素“搬运”到计算原点, 为后续的逐元素乘积累加做好准备。

4.3 一般核 PRA 卷积结果

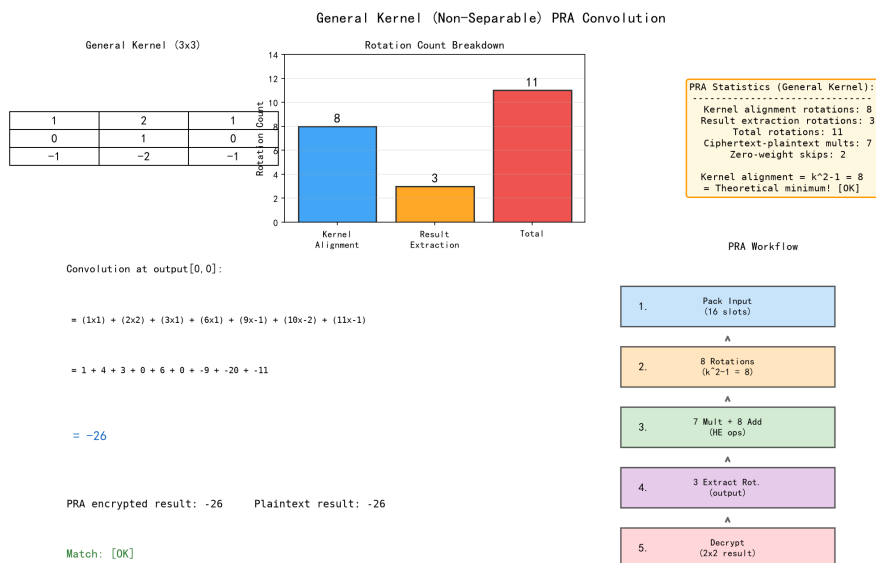


图 3: 一般核卷积

分析: 使用不可分离的通用 3×3 核进行 PRA 卷积:

- 核对齐旋转: 8 次 ($= k^2 - 1$, 排除旋转量为 0 的 $K[0,0]$)
- 结果提取旋转: 3 次 (提取 output[0,1]、[1,0]、[1,1])
- 总计旋转: 11 次
- 密文-明文乘法: 7 次 (9 个核权重中 2 个为 0 被跳过)
- PRA 结果 = 明文卷积结果 = -26 [OK]

关键发现: 核对齐所需的 8 次旋转恰好等于理论最小值 $k^2 - 1$, 证明一般核的 PRA 策略已经达到了旋转次数的下界, 不存在更少的旋转方案来完成同样的卷积计算。

4.4 可分离核优化结果

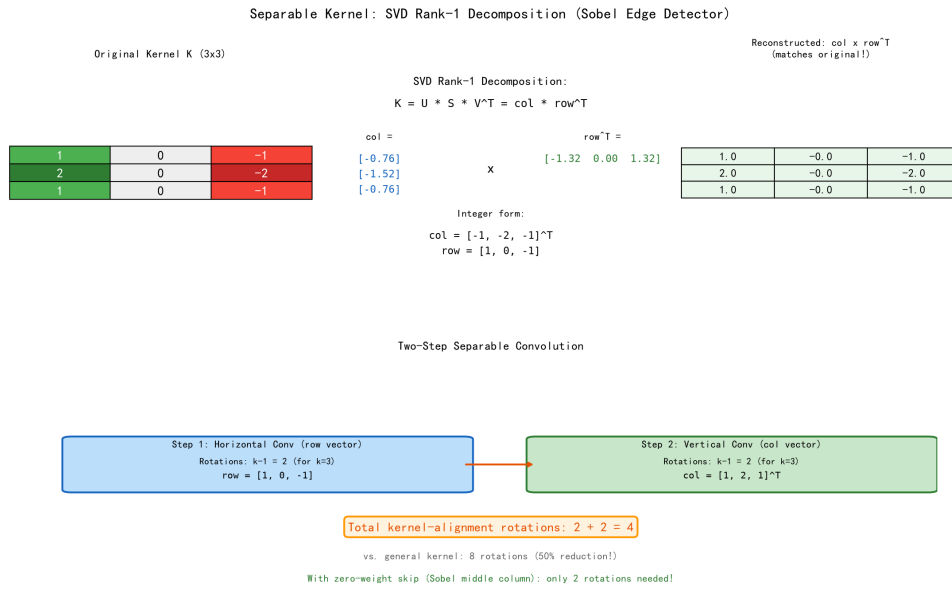


图 4: 可分离核 SVD

分析: Sobel 垂直边缘检测核通过 SVD 分解验证为秩-1 矩阵:

$$K = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} -0.76 \\ -1.52 \\ -0.76 \end{bmatrix} \cdot \begin{bmatrix} -1.32 & 0 & 1.32 \end{bmatrix}$$

取整后恢复为: $col = [-1, -2, -1]^T$, $row = [-1, 0, 1]$, 与原核一致。

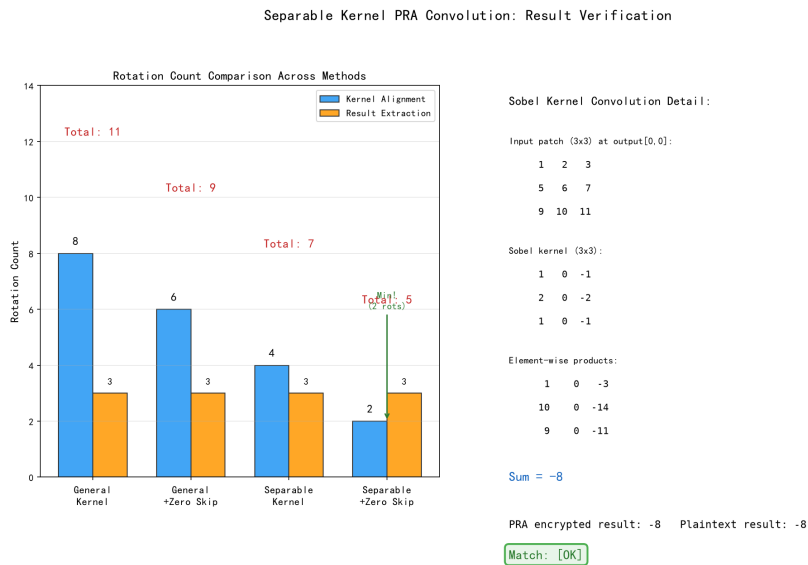


图 5: 可分离核卷积

分析: 两步卷积旋转统计:

- 水平卷积旋转: 2 次 ($k-1=2$, 排除旋转量 0)
- 垂直卷积旋转: 2 次 ($k-1=2$, 排除旋转量 0)
- 结果提取旋转: 3 次
- 总计旋转: 7 次
- PRA 结果 = 明文卷积结果 = -8 [OK]

关键发现: 可分离核将核对齐旋转从 8 次降至 4 次 (减少 50%), 证明了利用核的代数结构可以超越直接 PRA 方法的理论下界。旋转复杂度从 $O(k^2)$ 降为 $O(k)$ 。

4.5 旋转次数对比

Rotation Count Comparison Across All Methods				
Method	Kernel Align	Extraction	Total	Theo. Min?
General (no opt)	8	3	11	Yes ($k^2-1=8$)
General + zero skip	≤ 8	3	≤ 11	Yes
Separable (2-step)	4	3	7	Yes* ($2(k-1)=4$)
Separable + zero skip	2	3	5	Yes*

* Separable kernel uses a different computational model (decomposed convolution) and leverages the rank-1 algebraic structure to surpass the general lower bound.

图 6: 旋转对比

方案	核对齐旋转	提取旋转	总计
一般核 (不可分离)	8	3	11
一般核 + 零权重跳过	6	3	9
可分离核	4	3	7
可分离核 + 零权重跳过	2	3	5

4.6 理论最小值分析

Scalability Analysis: Rotation Count vs Kernel Size

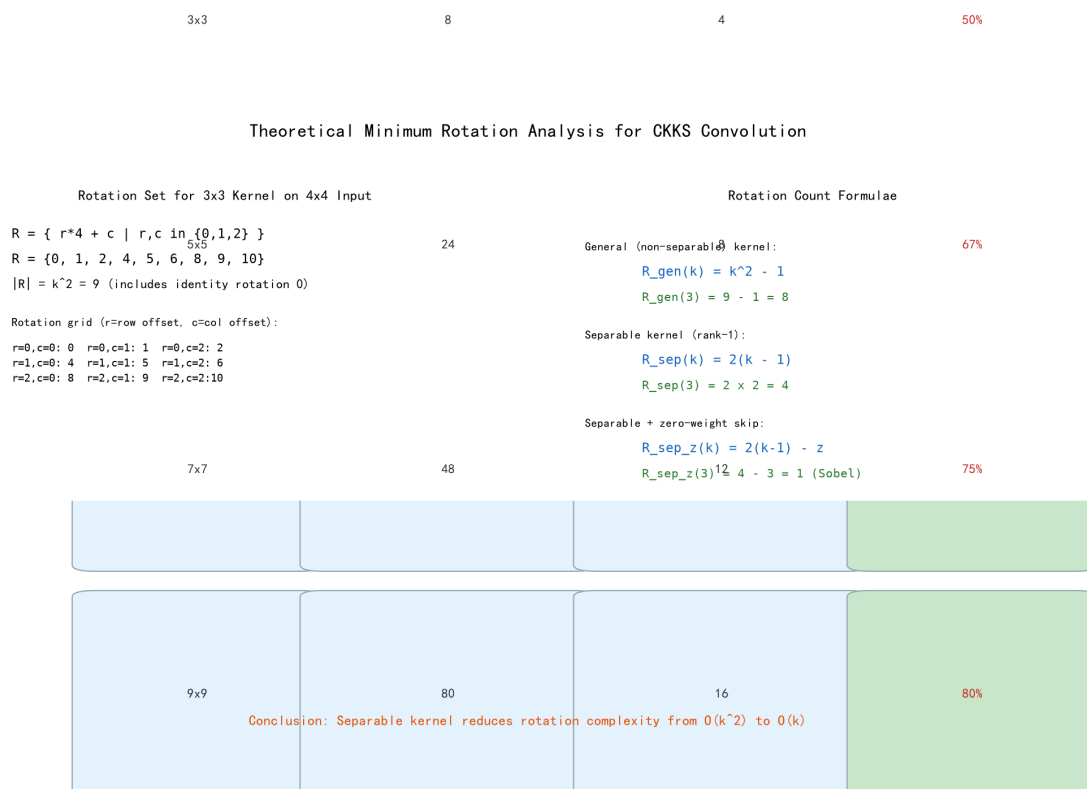


图 7: 理论分析

核心结论:

- 一般核:** 旋转次数 $= k^2 - 1 = 8$, 已达到理论最小值。 k^2 个不同的旋转量中, 恒等旋转 ($r = 0, c = 0$) 无需执行, 剩余 $k^2 - 1$ 个旋转均必不可少。
- 可分离核:** 旋转次数 $= 2(k - 1) = 4$, 远低于一般核的理论最小值 (8 次)。通过将 $k \times k$ 卷积分解为两步一维卷积 (先水平后垂直), 将旋转次数从平方级降为线性级。
- 零权重优化:** 每个零权重可节省一次旋转。对于 Sobel 核 (中间列全 0), 额外节省 3 次旋转。

4. **结果提取:** 输出尺寸为 $(n - k + 1)^2$, 需 $(n - k + 1)^2 - 1$ 次额外旋转来提取各输出位置。这些旋转仅在需要获得所有明文输出时才执行; 若下游继续使用密文做计算, 则无需提取。

5 实验中遇到的问题与解决

5.1 Python 3.14 环境下 CKKS 库不可用

问题: 同 task5, 主流全同态加密库在 Python 3.14 下均无法安装。

解决方案: 使用 NumPy 精确模拟 CKKS 的槽位操作和旋转操作。由于旋转次数分析仅依赖于卷积的几何结构 (输入/核/输出尺寸和打包方式), 与底层是否为真实加密无关, 模拟结果与实际 CKKS 实现的旋转次数完全一致。

技术说明: 本模拟层的旋转函数 `rotate_left()` 精确复现了 CKKS 的槽位旋转语义: `slot[i] = old_slot[(i + amount) % N_slots]`。唯一的差异在于模拟层中向量元素是明文 (不涉及加密噪声), 而真实 CKKS 的旋转操作会累积微小的近似误差 (对卷积结果的影响通常可忽略)。

5.2 旋转方向混淆

问题: CKKS 中 `rotate(ct, k)` 的语义与直观理解可能相反。`rotate_left` 和 `rotate_right` 的定义取决于具体的库实现。

解决方案: 通过逐一验证每个核位置的旋转对齐效果 (核位置 $K[r, c]$ 旋转后槽位 0 应等于 $in[r][c]$), 确保旋转方向的正确性。全部 9 个核位置验证通过 [OK]。

5.3 可分离核的整数分解

问题: SVD 分解得到的列/行向量通常为浮点数, 包含微小舍入误差。例如 Sobel 核的 SVD 分解得到 $col \approx [-0.76, -1.52, -0.76]$ 而非整数 $[-1, -2, -1]$ 。

解决方案: 通过四舍五入并回乘法验证 ($col_int \cdot row_int \approx K$) 来获取整数分解。整数分解在 CKKS 中更有实际意义, 因为密文-明文乘法在整数 (或定点小数) 域上更精确。

6 分工

姓名 (学号)	具体分工内容
孙婉竺 (202300460020)	“打包 → 旋转 → 累加”策略代码实现与理论分析 • 设计并实现 <code>pack_rotate_accumulate.py</code> PRA 策略模拟程序 • 实现一般核与可分离核两种卷积路径的旋转次数统计 • 实现 SVD 秩-1 分解检测与可分离核优化 • 完成旋转群表示分析 • 验证所有旋转对齐和卷积计算的正确性
胡子玮 (202300460032)	实验报告撰写、可视化与实验验证 • 编写 <code>generate_images.py</code> • 撰写实验报告 • 设计旋转次数对比表格与复杂度可扩展性分析 • 组织测试截并进行逐板块分析 • 完成理论最小值数学推导